

Pandas Handbook

pandas #python #datascience

Why Pandas?

Task	Without Pandas	With Pandas
Load CSV	<code>open()</code> + loops	<code>pd.read_csv()</code>
Filter rows	Custom loop	<code>df[df["col"] > 5]</code>
Group & summarize	Manual	<code>df.groupby()</code>
Merge datasets	Nested loops	<code>pd.merge()</code>

[!note] Pandas is built on top of NumPy — bridging NumPy performance with Excel-like usability.

Core Data Structures

Series — 1D Labeled Array

```
s = pd.Series([10, 20, 30]) # Auto index: 0, 1, 2
s = pd.Series([10, 20, 30], index=["a", "b", "c"]) # Custom index
```

DataFrame — 2D Labeled Table

```
data = {"name": ["Alice", "Bob", "Charlie"], "age": [25, 30, 35], "city": ["Delhi", "Mumbai", "Bangalore"]}
df = pd.DataFrame(data)
```

	name	age	city
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	35	Bangalore

Each column in a DataFrame is a Series.

```
df.index = ["a", "b"]          # Change row labels
df.columns = ["Name", "Age"]    # Change column labels
```

Creating DataFrames

```
# From list of lists
df = pd.DataFrame([["Alice", 25], ["Bob", 30]], columns=["Name", "Age"])

# From NumPy array
df = pd.DataFrame(np.array([[1,2],[3,4]]), columns=["A","B"])

# From CSV / Excel / JSON / SQL
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx", sheet_name="Sales")
df = pd.read_json("data.json")
df = pd.read_sql("SELECT * FROM users", conn)

# From URL
df = pd.read_csv("https://raw.githubusercontent.com/.../tips.csv")
```

EDA — Quick Exploration

```
df.head()      # First 5 rows
df.tail()      # Last 5 rows
df.info()      # Column types + non-null counts
df.describe()  # Stats for numeric columns
df.shape       # (rows, columns)
df.columns     # Column names
```

Data Selection & Filtering

Selecting Data

```
df["Age"]      # Single column → Series
df[["Name", "Age"]] # Multiple columns → DataFrame

df.loc[0]      # Row by label
df.iloc[0]     # Row by position
df.loc[0, "Name"] # Single value (label)
df.iloc[0, 1]   # Single value (position)
df.loc[0:2, ["Name", "Age"]] # Slice rows + select columns

df.at[0, "Name"] # Fast single value (label)
df.iat[0, 1]     # Fast single value (position)
```

Filtering

```
df[df["Age"] > 30] # Simple condition
df[(df["Age"] > 25) & (df["City"] == "Delhi")] # AND
df[(df["Name"] == "Bob") | (df["Age"] < 30)] # OR
```

`.query()` — Cleaner Filtering

```
df.query("Age > 25 and City == 'Delhi'")    # ✅ use 'and', not '&'
df.query("25 < Age <= 40")                  # Chained comparison
df.query("`first name` == 'Alice'")         # Backticks for spaces in col names
df.query("Age > @age_limit")                # @ to reference Python variables
```

Data Cleaning

Missing Values

```
df.isnull().sum()                          # Count NaNs per column
df.dropna()                                # Drop rows with any NaN
df.dropna(axis=1)                           # Drop columns with NaN
df.fillna(0)                                # Fill NaN with 0
df["Age"].fillna(df["Age"].mean())          # Fill with column mean
df.ffill()                                  # Forward fill
df.bfill()                                  # Backward fill
```

Duplicates

```
df.duplicated()                             # Boolean mask of duplicates
df.drop_duplicates()                         # Remove duplicates
df.duplicated(subset=["Name", "Age"])       # Check specific columns
```

String Operations

```
df["Name"].str.lower()
df["City"].str.contains("delhi", case=False)
df["Email"].str.split("@")
# Chain: df["Name"].str.strip().str.upper()
```

Type Conversion

```
df["Age"] = df["Age"].astype(int)
df["Date"] = pd.to_datetime(df["Date"]) # Handles mixed formats, NaT, UNIX time
df["Cat"] = df["Cat"].astype("category")
df.dtypes                                # Check all types
```

Applying Functions

```
df["Age Group"] = df["Age"].apply(lambda x: "Adult" if x >= 18 else "Minor")
df["Gender"] = df["Gender"].map({"M": "Male", "F": "Female"})
df["City"].replace({"Del": "Delhi", "Mum": "Mumbai"})
```

Data Transformation

Sorting & Ranking

```
df.sort_values("Age")                                # Ascending
df.sort_values("Age", ascending=False)               # Descending
df.sort_values(["Age", "Salary"])                    # Multiple columns
df.sort_index()                                      # Sort by index
df.reset_index(drop=True, inplace=True)              # Reset to 0, 1, 2...

df["Rank"] = df["Score"].rank()                      # Average rank for ties → 1.5
df["Rank"] = df["Score"].rank(method="dense")        # No gaps → 1, 2, 2, 3
```

Renaming & Reordering

```
df.rename(columns={"oldName": "newName"}, inplace=True)
df.columns = ["Name", "Age", "City"] # Rename all

df = df[["City", "Name", "Age"]] # Reorder columns
cols = ["Name"] + [c for c in df.columns if c != "Name"] # Move col to front
df = df[cols]
```

Reshaping: melt & pivot

`melt()` — Wide → Long

Before (wide format):

Name	Math	Science	English
Alice	85	90	88
Bob	78	82	85
Charlie	92	89	94

```
df.melt(id_vars=["Name"],
        value_vars=["Math", "Science", "English"],
        var_name="Subject",
        value_name="Score")
```

After (long format):

Name	Subject	Score
Alice	Math	85
Alice	Science	90
Alice	English	88
Bob	Math	78
Bob	Science	82
...	...	...

Turns subject columns into rows — great for plotting & tidy data.

`pivot()` — Long → Wide

Reverse of melt. Takes the long format above back to wide.

```
df.pivot(index="Name", columns="Subject", values="Score")
```

Name	English	Math	Science
Alice	88	85	90
Bob	85	78	82
Charlie	94	92	89

[!warning] Raises an error if there are duplicate `index + column` combinations. Use `pivot_table()` instead.

`pivot_table()` — pivot with duplicates

```
df.pivot_table(index="Name", columns="Subject", values="Score", aggfunc="mean")
```

Name	Math	Science
Alice	82.5	90
Bob	80.0	NaN

Averages duplicate entries (e.g. two Math scores for Alice → 82.5). Empty combos show `NaN`.

Aggregation & Grouping

```
df.groupby("Department")["Salary"].mean()           # Avg salary per dept
df.groupby(["Team", "Gender"])["Salary"].mean()      # Multiple group keys

# Common aggregations
.sum() / .mean() / .count() / .min() / .max()

# Multiple functions at once
df.groupby("Team")["Salary"].agg(["mean", "max", "min"])

# Named aggregations
df.groupby("Team")["Salary"].agg(avg="mean", top="max")

# Different functions per column
df.groupby("Team").agg({"Salary": "mean", "Age": "max"})
```

transform vs filter

```
# .transform() – adds group result back to original rows
df["Team Avg"] = df.groupby("Team")["Salary"].transform("mean")

# .filter() – keeps only groups meeting a condition
df.groupby("Team").filter(lambda x: x["Salary"].mean() > 80)
```

Method	Returns	Use For
<code>.agg()</code>	Single value per group	Summary stats
<code>.transform()</code>	Same shape as original	Add group-based column
<code>.filter()</code>	Subset of rows	Keep/drop whole groups

Merging & Joining

Sample data — `employees` :

EmpID	Name	DeptID
1	Alice	10
2	Bob	20
3	Charlie	30

Sample data — `departments` :

DeptID	DeptName
10	HR
20	Engineering
40	Marketing

```
# Inner join (default) – only matching keys
pd.merge(employees, departments, on="DeptID")
```

EmpID	Name	DeptID	DeptName
1	Alice	10	HR
2	Bob	20	Engineering

```
# Left join – all rows from left, NaN where no match
pd.merge(employees, departments, on="DeptID", how="left")
```

EmpID	Name	DeptID	DeptName
1	Alice	10	HR
2	Bob	20	Engineering
3	Charlie	30	NaN

```
# Right join – all rows from right, NaN where no match
pd.merge(employees, departments, on="DeptID", how="right")
```

EmpID	Name	DeptID	DeptName
1	Alice	10	HR
2	Bob	20	Engineering
NaN	NaN	40	Marketing

```
# Outer join – all rows from both, NaN where missing
pd.merge(employees, departments, on="DeptID", how="outer")
```

EmpID	Name	DeptID	DeptName
1	Alice	10	HR
2	Bob	20	Engineering
3	Charlie	30	NaN
NaN	NaN	40	Marketing

```
# Concat vertically (stack rows)
pd.concat([df1, df2])

# Concat horizontally (stack columns)
pd.concat([df1, df2], axis=1)
```

Use Case	Method
SQL-style joins	<code>pd.merge()</code>
Stack rows	<code>pd.concat([df1, df2])</code>
Stack columns	<code>pd.concat([df1, df2], axis=1)</code>

Reading & Writing Files

```
# CSV
df = pd.read_csv("data.csv", usecols=["Name", "Age"], nrows=10)
df.to_csv("output.csv", index=False)

# Excel
df = pd.read_excel("data.xlsx", sheet_name="Sales")
df.to_excel("output.xlsx", index=False)

# Multiple sheets
with pd.ExcelWriter("report.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Summary", index=False)
    df2.to_excel(writer, sheet_name="Details", index=False)

# JSON
df = pd.read_json("data.json")
```